

Setup

```
# importing libraries
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import sys

pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:.4f}")

plt.rcParams["figure.figsize"] = (10, 6)

# path vars and files
PROJECT_ROOT = Path.cwd().resolve().parents[1]

DATA_DIR = PROJECT_ROOT / "data" / "raw"

TRAIN_PATH = DATA_DIR / "UNSW_NB15_training-set.csv"
TEST_PATH = DATA_DIR / "UNSW_NB15_testing-set.csv"
FEATURES_PATH = DATA_DIR / "NUSW-NB15_features.csv"

sys.path.insert(0, str(PROJECT_ROOT))

print("Project root:", PROJECT_ROOT)
print("Data directory:", DATA_DIR)

print("\nFiles found:")
for file in DATA_DIR.glob("*.csv"):
    print("-", file.name)

Project root: C:\Projects\Aeges-Q
Data directory: C:\Projects\Aeges-Q\data\raw

Files found:
- NUSW-NB15_features.csv
- UNSW-NB15_1.csv
- UNSW-NB15_2.csv
- UNSW-NB15_3.csv
- UNSW-NB15_4.csv
- UNSW-NB15_LIST_EVENTS.csv
- UNSW_NB15_testing-set.csv
- UNSW_NB15_training-set.csv
```

```
# train and test dataframes
```

```
train_df = pd.read_csv(TRAIN_PATH)
```

```
test_df = pd.read_csv(TEST_PATH)
```

```
print("Training shape:", train_df.shape)
```

```
print("Testing shape:", test_df.shape)
```

```
Training shape: (82332, 45)
```

```
Testing shape: (175341, 45)
```

```
# Display the first few rows of the training dataframe
```

```
train_df.head()
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes
rate \									
0	1	0.0000	udp	-	INT	2	0	496	0
90909.0902									
1	2	0.0000	udp	-	INT	2	0	1762	0
125000.0003									
2	3	0.0000	udp	-	INT	2	0	1068	0
200000.0051									
3	4	0.0000	udp	-	INT	2	0	900	0
166666.6608									
4	5	0.0000	udp	-	INT	2	0	2126	0
100000.0025									

	sttl	dttl	sload	dload	sloss	dloss	sinpkt	dinpkt
sjit \								
0	254	0	180363632.0000	0.0000	0	0	0.0110	0.0000
0.0000								
1	254	0	881000000.0000	0.0000	0	0	0.0080	0.0000
0.0000								
2	254	0	854400000.0000	0.0000	0	0	0.0050	0.0000
0.0000								
3	254	0	600000000.0000	0.0000	0	0	0.0060	0.0000
0.0000								
4	254	0	850400000.0000	0.0000	0	0	0.0100	0.0000
0.0000								

	djit	swin	stcpb	dtcpb	dwin	tcprtt	synack	ackdat	smean
dmean \									
0	0.0000	0	0	0	0	0.0000	0.0000	0.0000	248
0									
1	0.0000	0	0	0	0	0.0000	0.0000	0.0000	881
0									
2	0.0000	0	0	0	0	0.0000	0.0000	0.0000	534
0									
3	0.0000	0	0	0	0	0.0000	0.0000	0.0000	450
0									
4	0.0000	0	0	0	0	0.0000	0.0000	0.0000	1063

0

	trans_depth	response_body_len	ct_srv_src	ct_state_ttl
ct_dst_ltm \				
0	0	0	2	2
1				
1	0	0	2	2
1				
2	0	0	3	2
1				
3	0	0	3	2
2				
4	0	0	3	2
2				

	ct_src_dport_ltm	ct_dst_sport_ltm	ct_dst_src_ltm	is_ftp_login	\
0	1	1	2	0	
1	1	1	2	0	
2	1	1	3	0	
3	2	1	3	0	
4	2	1	3	0	

	ct_ftp_cmd	ct_flw_http_mthd	ct_src_ltm	ct_srv_dst
is_sm_ips_ports \				
0	0	0	1	2
0				
1	0	0	1	2
0				
2	0	0	1	3
0				
3	0	0	2	3
0				
4	0	0	2	3
0				

	attack_cat	label
0	Normal	0
1	Normal	0
2	Normal	0
3	Normal	0
4	Normal	0

columns

print(train_df.columns.tolist())

```
['id', 'dur', 'proto', 'service', 'state', 'spkts', 'dpkts', 'sbytes',  
'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload', 'sloss', 'dloss',  
'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb', 'dwin',  
'tcprtt', 'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',  
'response_body_len', 'ct_srv_src', 'ct_state_ttl', 'ct_dst_ltm',
```

```
'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm',  
'is_ftp_login', 'ct_ftp_cmd', 'ct_flw_http_mthd', 'ct_src_ltm',  
'ct_srv_dst', 'is_sm_ips_ports', 'attack_cat', 'label']
```

```
# training data information
```

```
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 82332 entries, 0 to 82331
```

```
Data columns (total 45 columns):
```

#	Column	Non-Null Count	Dtype
0	id	82332 non-null	int64
1	dur	82332 non-null	float64
2	proto	82332 non-null	object
3	service	82332 non-null	object
4	state	82332 non-null	object
5	spkts	82332 non-null	int64
6	dpkts	82332 non-null	int64
7	sbytes	82332 non-null	int64
8	dbytes	82332 non-null	int64
9	rate	82332 non-null	float64
10	sttl	82332 non-null	int64
11	dttl	82332 non-null	int64
12	sload	82332 non-null	float64
13	dload	82332 non-null	float64
14	sloss	82332 non-null	int64
15	dloss	82332 non-null	int64
16	sinpkt	82332 non-null	float64
17	dinpkt	82332 non-null	float64
18	sjit	82332 non-null	float64
19	djit	82332 non-null	float64
20	swin	82332 non-null	int64
21	stcpb	82332 non-null	int64
22	dtcpb	82332 non-null	int64
23	dwin	82332 non-null	int64
24	tcprtt	82332 non-null	float64
25	synack	82332 non-null	float64
26	ackdat	82332 non-null	float64
27	smean	82332 non-null	int64
28	dmean	82332 non-null	int64
29	trans_depth	82332 non-null	int64
30	response_body_len	82332 non-null	int64
31	ct_srv_src	82332 non-null	int64
32	ct_state_ttl	82332 non-null	int64
33	ct_dst_ltm	82332 non-null	int64
34	ct_src_dport_ltm	82332 non-null	int64
35	ct_dst_sport_ltm	82332 non-null	int64
36	ct_dst_src_ltm	82332 non-null	int64
37	is_ftp_login	82332 non-null	int64

```
38 ct_ftp_cmd      82332 non-null int64
39 ct_flw_http_mthd 82332 non-null int64
40 ct_src_ltm       82332 non-null int64
41 ct_srv_dst       82332 non-null int64
42 is_sm_ips_ports  82332 non-null int64
43 attack_cat       82332 non-null object
44 label            82332 non-null int64
dtypes: float64(11), int64(30), object(4)
memory usage: 28.3+ MB
```

Dataset Integrity Audit

```
# verifying identical schemas
print(
    "Schemas identical:",
    list(train_df.columns) == list(test_df.columns)
)

print("\nColumns only in train:")
print(set(train_df.columns) - set(test_df.columns))

print("\nColumns only in test:")
print(set(test_df.columns) - set(train_df.columns))

Schemas identical: True

Columns only in train:
set()

Columns only in test:
set()

# inspecting the tables
print("Binary label distribution:\n")
print(train_df["label"].value_counts())

Binary label distribution:

label
1    45332
0    37000
Name: count, dtype: int64

# binary label percentages
print("\nBinary label percentages:\n")
print(
    train_df["label"]
    .value_counts(normalize=True)
    .mul(100)
```

```
)  
    .round(2)  
)
```

Binary label percentages:

```
label  
1    55.0600  
0    44.9400  
Name: proportion, dtype: float64
```

```
# inspecting attack categories  
print("\nAttack category distribution:\n")  
  
print(  
    train_df["attack_cat"]  
    .value_counts()  
)
```

Attack category distribution:

```
attack_cat  
Normal          37000  
Generic         18871  
Exploits        11132  
Fuzzers         6062  
DoS             4089  
Reconnaissance  3496  
Analysis        677  
Backdoor        583  
Shellcode       378  
Worms           44  
Name: count, dtype: int64
```

```
pd.crosstab(  
    train_df["attack_cat"],  
    train_df["label"]  
)
```

label	0	1
attack_cat		
Analysis	0	677
Backdoor	0	583
DoS	0	4089
Exploits	0	11132
Fuzzers	0	6062
Generic	0	18871
Normal	37000	0
Reconnaissance	0	3496

Shellcode	0	378
Worms	0	44

missing values

```
train_missing = train_df.isnull().sum()  
test_missing = test_df.isnull().sum()
```

```
print("Train missing values:")  
display(  
    train_missing[train_missing > 0]  
    .sort_values(ascending=False)  
)
```

```
print("\nTest missing values:")  
display(  
    test_missing[test_missing > 0]  
    .sort_values(ascending=False)  
)
```

Train missing values:

Series([], dtype: int64)

Test missing values:

Series([], dtype: int64)

duplicate rows

```
print(  
    "Duplicate rows in train:",  
    train_df.duplicated().sum()  
)
```

```
print(  
    "Duplicate rows in test:",  
    test_df.duplicated().sum()  
)
```

Duplicate rows in train: 0

Duplicate rows in test: 0

duplicates excluding id column

```
feature_columns = [col for col in train_df.columns if col != "id"]
```

```
print(  
    "Feature duplicates in train:",  
    train_df[feature_columns].duplicated().sum()  
)
```

```
print(  
    "Feature duplicates in test:",
```

```
test_df[feature_columns].duplicated().sum()  
)
```

Feature duplicates in train: 26387

Feature duplicates in test: 67601

unique values per column

```
unique_counts = train_df.nunique().sort_values()
```

unique_counts

is_sm_ips_ports	2
label	2
is_ftp_login	3
ct_ftp_cmd	3
state	7
ct_state_ttl	7
trans_depth	8
dttl	8
ct_flw_http_mthd	8
attack_cat	10
swin	11
sttl	11
service	13
dwin	14
ct_dst_sport_ltm	33
ct_dst_ltm	50
ct_src_dport_ltm	50
ct_src_ltm	50
ct_dst_src_ltm	57
ct_srv_dst	57
ct_srv_src	57
proto	131
sloss	253
dloss	311
spkts	420
dpkts	436
response_body_len	1190
dmean	1222
smean	1282
dbytes	4034
sbytes	4489
ackdat	24020
synack	24934
tcprrt	26130
dinpkt	37617
djit	38381
dtcpb	39108
stcpb	39219
dur	39888

```
sjit          39944
sinpkt        39970
dload         40614
rate          40616
sload         42873
id            82332
dtype: int64
```

```
low_cardinality = unique_counts[
    unique_counts <= 20
]
```

```
low_cardinality
```

```
is_sm_ips_ports    2
label              2
is_ftp_login       3
ct_ftp_cmd         3
state              7
ct_state_ttl       7
trans_depth        8
dttl               8
ct_flw_http_mthd   8
attack_cat         10
swin               11
sttl               11
service            13
dwin               14
dtype: int64
```

```
# constant features
```

```
low_cardinality = unique_counts[
    unique_counts <= 20
]
```

```
low_cardinality
```

```
is_sm_ips_ports    2
label              2
is_ftp_login       3
ct_ftp_cmd         3
state              7
ct_state_ttl       7
trans_depth        8
dttl               8
ct_flw_http_mthd   8
attack_cat         10
swin               11
sttl               11
service            13
```

```
dwin          14
dtype: int64
```

Class and Attack Distribution

```
# Binary class distribution

label_counts = train_df["label"].value_counts().sort_index()

print(label_counts)

label_percentages = (
    train_df["label"]
    .value_counts(normalize=True)
    .sort_index()
    * 100
)

print("\nPercentages:")
print(label_percentages.round(2))

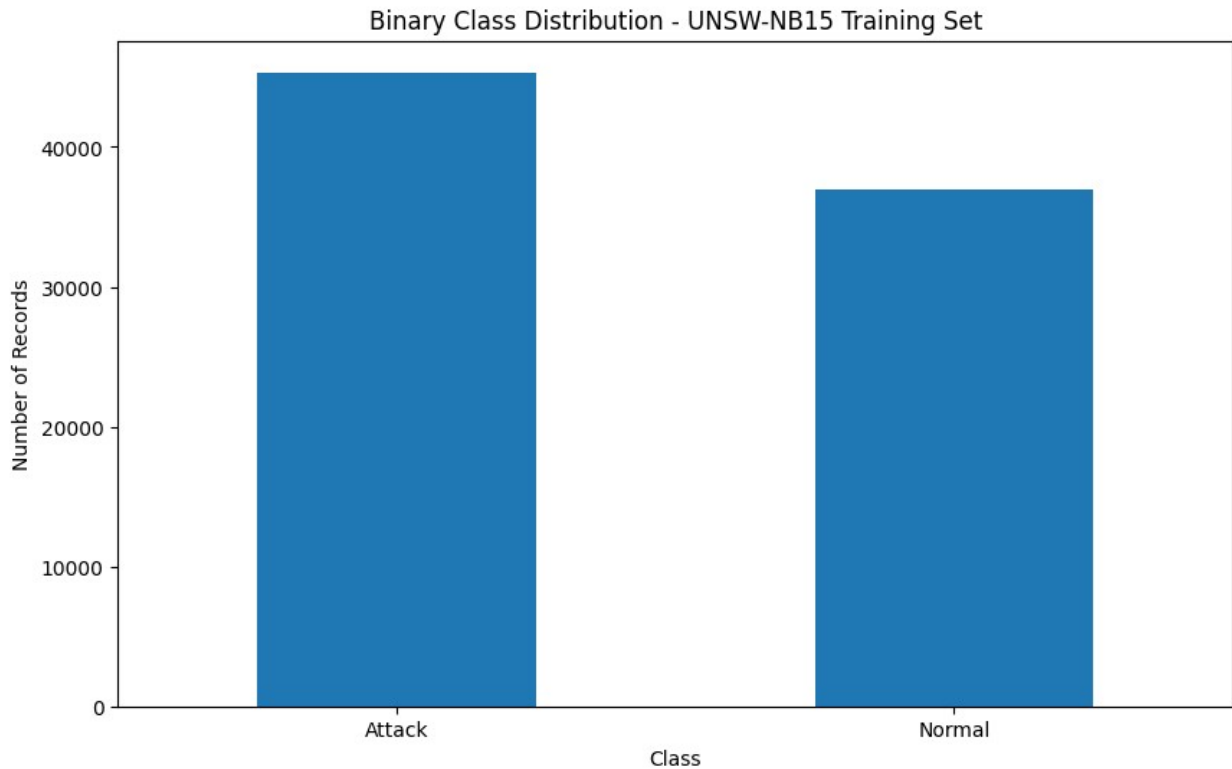
label
0    37000
1    45332
Name: count, dtype: int64

Percentages:
label
0    44.9400
1    55.0600
Name: proportion, dtype: float64

label_names = train_df["label"].map({
    0: "Normal",
    1: "Attack"
})

label_names.value_counts().plot(
    kind="bar",
    title="Binary Class Distribution - UNSW-NB15 Training Set",
    xlabel="Class",
    ylabel="Number of Records"
)

plt.xticks(rotation=0)
plt.show()
```



```
# Attack Category Distribution
attack_counts = train_df["attack_cat"].value_counts()

print(attack_counts)

attack_cat
Normal      37000
Generic     18871
Exploits    11132
Fuzzers      6062
DoS          4089
Reconnaissance 3496
Analysis      677
Backdoor      583
Shellcode     378
Worms         44
Name: count, dtype: int64

attack_percentages = (
    train_df["attack_cat"]
    .value_counts(normalize=True)
    * 100
)

print(attack_percentages.round(2))
```

```

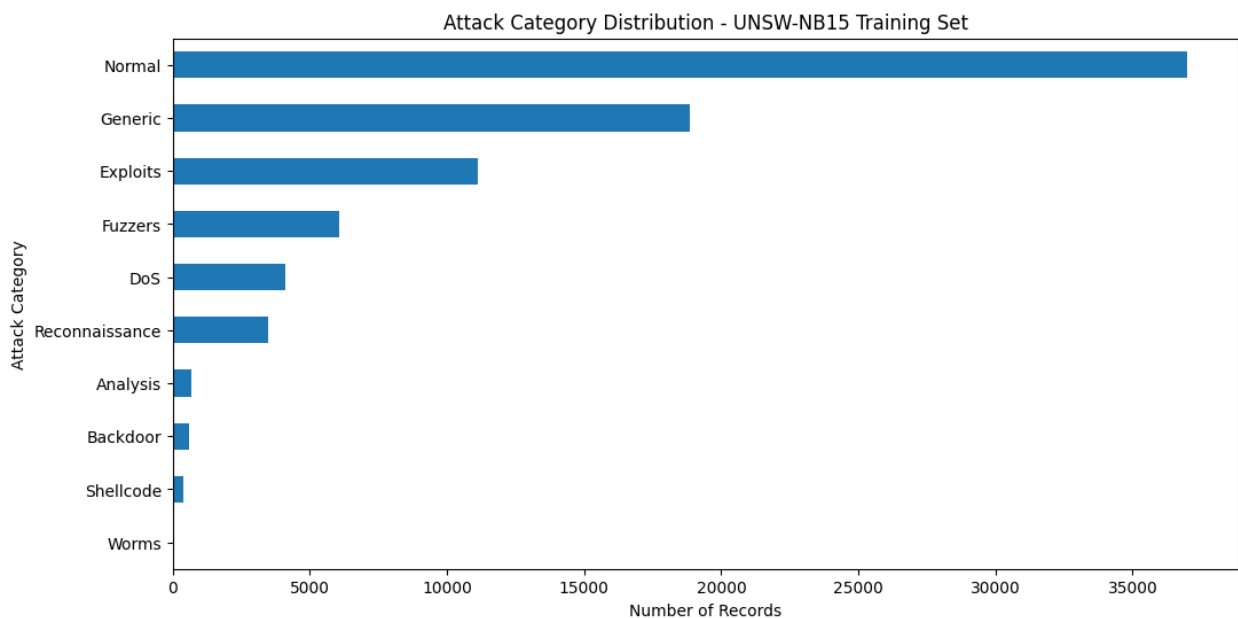
attack_cat
Normal      44.9400
Generic     22.9200
Exploits    13.5200
Fuzzers      7.3600
DoS          4.9700
Reconnaissance 4.2500
Analysis     0.8200
Backdoor     0.7100
Shellcode    0.4600
Worms        0.0500
Name: proportion, dtype: float64

plt.figure(figsize=(12, 6))

attack_counts.sort_values().plot(
    kind="barh",
    title="Attack Category Distribution - UNSW-NB15 Training Set",
    xlabel="Number of Records",
    ylabel="Attack Category"
)

plt.show()

```



```

# Relationship Between Attack_Cat and Label
pd.crosstab(
    train_df["attack_cat"],
    train_df["label"]
)

```

label	0	1
attack_cat		
Analysis	0	677
Backdoor	0	583
DoS	0	4089
Exploits	0	11132
Fuzzers	0	6062
Generic	0	18871
Normal	37000	0
Reconnaissance	0	3496
Shellcode	0	378
Worms	0	44

dataset summary

```
def dataset_summary(df, name):
    print(f"\n{'=' * 50}")
    print(name)
    print(f"{'=' * 50}")

    print("\nShape:")
    print(df.shape)

    print("\nBinary label distribution:")
    print(df["label"].value_counts())

    print("\nBinary label percentage:")
    print(
        (df["label"]
         .value_counts(normalize=True)
         * 100)
        .round(2)
    )

    print("\nAttack categories:")
    print(df["attack_cat"].value_counts())
```

dataset_summary(train_df, "TRAINING SET")

dataset_summary(test_df, "TESTING SET")

=====

TRAINING SET

=====

Shape:
(82332, 45)

Binary label distribution:
label

```
1    45332
0    37000
Name: count, dtype: int64
```

```
Binary label percentage:
label
```

```
1    55.0600
0    44.9400
Name: proportion, dtype: float64
```

```
Attack categories:
```

```
attack_cat
Normal          37000
Generic         18871
Exploits        11132
Fuzzers          6062
DoS              4089
Reconnaissance  3496
Analysis         677
Backdoor         583
Shellcode        378
Worms            44
Name: count, dtype: int64
```

```
=====
TESTING SET
=====
```

```
Shape:
(175341, 45)
```

```
Binary label distribution:
```

```
label
1    119341
0     56000
Name: count, dtype: int64
```

```
Binary label percentage:
label
```

```
1    68.0600
0    31.9400
Name: proportion, dtype: float64
```

```
Attack categories:
```

```
attack_cat
Normal          56000
Generic         40000
Exploits        33393
Fuzzers         18184
DoS             12264
```

```
Reconnaissance    10491
Analysis          2000
Backdoor          1746
Shellcode         1133
Worms             130
Name: count, dtype: int64
```

Understanding the Features

2. Feature Type Analysis

The dataset contains:

- Identifier: `id`
- Categorical network features: `proto`, `service`, `state`
- Numerical network-flow features
- `attack_cat`: attack category metadata
- `label`: binary target

```
categorical_features = [
    "proto",
    "service",
    "state"
]

target_columns = [
    "attack_cat",
    "label"
]

id_column = "id"

numerical_features = [
    col for col in train_df.columns
    if col not in categorical_features + target_columns + [id_column]
]

print("Categorical features:", categorical_features)
print("Number of numerical features:", len(numerical_features))

Categorical features: ['proto', 'service', 'state']
Number of numerical features: 39

# inspect categorical features
for col in categorical_features:
    print(f"\n{'=' * 50}")
    print(f"{col.upper()}")
    print(f"{'=' * 50}")
```

```
print("Unique values:", train_df[col].nunique())
print(train_df[col].value_counts().head(20))
```

```
=====
PROTO
=====
```

```
Unique values: 131
```

```
proto
```

```
tcp          43095
```

```
udp          29418
```

```
unas         3515
```

```
arp           987
```

```
ospf          676
```

```
sctp          324
```

```
any            96
```

```
gre            88
```

```
rsvp           64
```

```
ipv6           61
```

```
sep            58
```

```
sun-nd         54
```

```
swipe          52
```

```
mobile         52
```

```
pim            52
```

```
xns-idp        34
```

```
leaf-1         34
```

```
bbn-rcc         34
```

```
prm            34
```

```
irtp           33
```

```
Name: count, dtype: int64
```

```
=====
SERVICE
=====
```

```
Unique values: 13
```

```
service
```

```
-          47153
```

```
dns        21367
```

```
http        8287
```

```
smtp        1851
```

```
ftp         1552
```

```
ftp-data    1396
```

```
pop3         423
```

```
ssh          204
```

```
ssl           30
```

```
snmp          29
```

```
dhcp          26
```

```
radius         9
```

```
irc            5
```

```
Name: count, dtype: int64
```

```
=====
```

```
STATE
```

```
=====
```

```
Unique values: 7
```

```
state
```

```
FIN      39339
```

```
INT      34163
```

```
CON       6982
```

```
REQ      1842
```

```
ACC         4
```

```
RST         1
```

```
CLO         1
```

```
Name: count, dtype: int64
```

```
fig, axes = plt.subplots(3, 1, figsize=(14, 18))
```

```
for ax, col in zip(axes, categorical_features):  
    train_df[col].value_counts().head(20).plot(  
        kind="bar",  
        ax=ax  
    )
```

```
    ax.set_title(f"Top Categories in {col}")
```

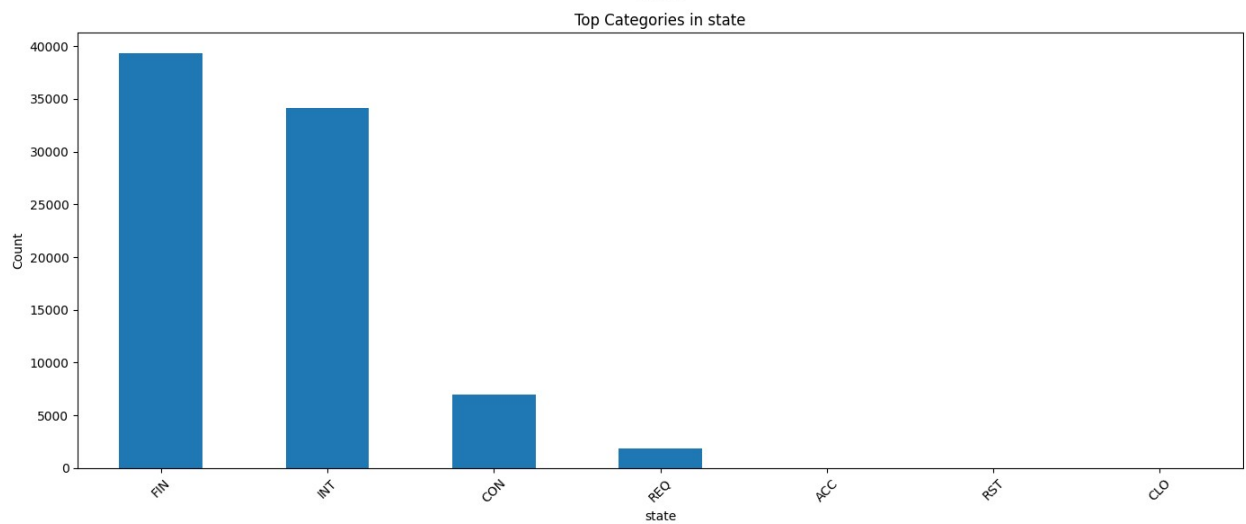
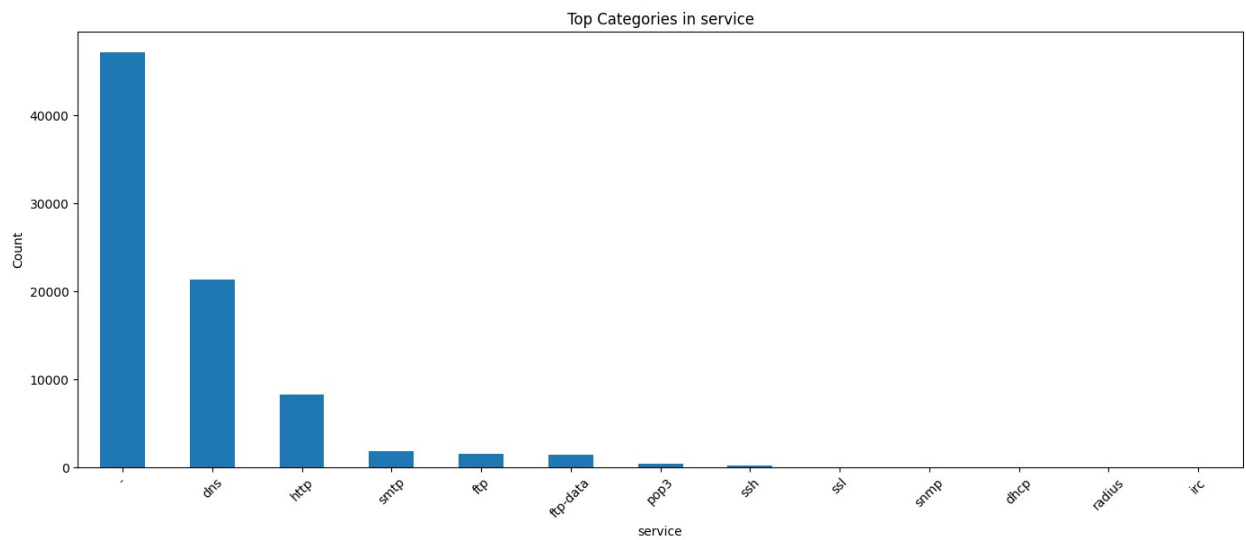
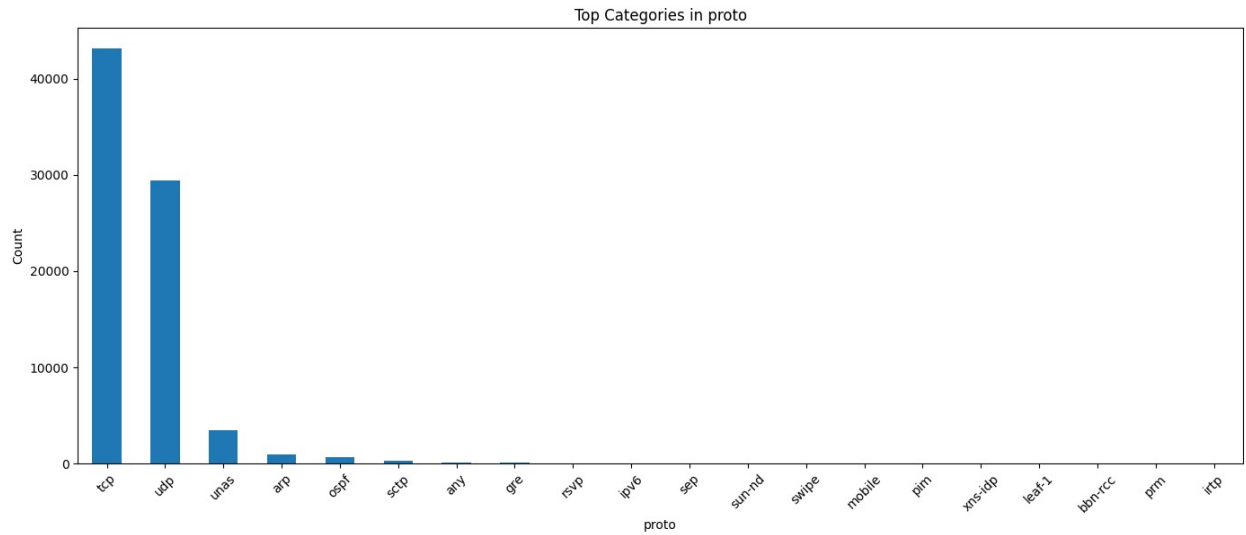
```
    ax.set_xlabel(col)
```

```
    ax.set_ylabel("Count")
```

```
    ax.tick_params(axis="x", rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
# numerical feature summary
numerical_summary = train_df[numerical_features].describe().T
```

numerical_summary

	count	mean	std	
min \				
dur	82332.0000	1.0068	4.7104	0.0000
spkts	82332.0000	18.6665	133.9164	1.0000
dpkts	82332.0000	17.5459	115.5741	0.0000
sbytes	82332.0000	7993.9082	171642.2619	24.0000
dbytes	82332.0000	13233.7856	151471.4561	0.0000
rate	82332.0000	82410.8867	148620.3670	0.0000
sttl	82332.0000	180.9677	101.5134	0.0000
dttl	82332.0000	95.7130	116.6677	0.0000
sload	82332.0000	64549016.9141	179861832.6300	0.0000
dload	82332.0000	630546.9590	2393000.5556	0.0000
sloss	82332.0000	4.7537	64.6496	0.0000
dloss	82332.0000	6.3086	55.7080	0.0000
sinpkt	82332.0000	755.3943	6182.6157	0.0000
dinpkt	82332.0000	121.7013	1292.3785	0.0000
sjit	82332.0000	6363.0751	56724.0167	0.0000
djit	82332.0000	535.1804	3635.3054	0.0000
swin	82332.0000	133.4591	127.3570	0.0000
stcpb	82332.0000	1084641551.1153	1390859761.6104	0.0000
dtcpb	82332.0000	1073464670.8804	1381996192.0328	0.0000
dwin	82332.0000	128.2866	127.4914	0.0000
tcprtt	82332.0000	0.0559	0.1160	0.0000
synack	82332.0000	0.0293	0.0709	0.0000
ackdat	82332.0000	0.0267	0.0551	0.0000
smean	82332.0000	139.5286	208.4721	24.0000

dmean	82332.0000	116.2751	244.6003	0.0000
trans_depth	82332.0000	0.0943	0.5429	0.0000
response_body_len	82332.0000	1595.3719	38066.9723	0.0000
ct_srv_src	82332.0000	9.5466	11.0903	1.0000
ct_state_ttl	82332.0000	1.3693	1.0672	0.0000
ct_dst_ltm	82332.0000	5.7449	8.4181	1.0000
ct_src_dport_ltm	82332.0000	4.9289	8.3895	1.0000
ct_dst_sport_ltm	82332.0000	3.6630	5.9154	1.0000
ct_dst_src_ltm	82332.0000	7.4564	11.4152	1.0000
is_ftp_login	82332.0000	0.0083	0.0912	0.0000
ct_ftp_cmd	82332.0000	0.0084	0.0925	0.0000
ct_flw_http_mthd	82332.0000	0.1297	0.6387	0.0000
ct_src_ltm	82332.0000	6.4684	8.5439	1.0000
ct_srv_dst	82332.0000	9.1643	11.1214	1.0000
is_sm_ips_ports	82332.0000	0.0111	0.1049	0.0000
	25%	50%	75%	
max				
dur	0.0000	0.0141	0.7194	
60.0000				
spkts	2.0000	6.0000	12.0000	
10646.0000				
dpkts	0.0000	2.0000	10.0000	
11018.0000				
sbytes	114.0000	534.0000	1280.0000	
14355774.0000				
dbytes	0.0000	178.0000	956.0000	
14657531.0000				
rate	28.6061	2650.1767	111111.1072	
1000000.0030				
sttl	62.0000	254.0000	254.0000	
255.0000				
dttl	0.0000	29.0000	252.0000	
253.0000				
sload	11202.4668	577003.2188	65142856.0000	

5268000256.0000			
dload	0.0000	2112.9514	15858.0823
20821108.0000			
sloss	0.0000	1.0000	3.0000
5319.0000			
dloss	0.0000	0.0000	2.0000
5507.0000			
sinpkt	0.0080	0.5579	63.4094
60009.9920			
dinpkt	0.0000	0.0100	63.1364
57739.2400			
sjit	0.0000	17.6239	3219.3324
1483830.9170			
djit	0.0000	0.0000	128.4599
463199.2401			
swin	0.0000	255.0000	255.0000
255.0000			
stcpb	0.0000	27888855.0000	2171309606.5000
4294949667.0000			
dtcpb	0.0000	28569748.5000	2144205173.0000
4294880717.0000			
dwin	0.0000	255.0000	255.0000
255.0000			
tcprtt	0.0000	0.0006	0.1055
3.8215			
synack	0.0000	0.0004	0.0526
3.2268			
ackdat	0.0000	0.0001	0.0488
2.9288			
smean	57.0000	65.0000	100.0000
1504.0000			
dmean	0.0000	44.0000	87.0000
1500.0000			
trans_depth	0.0000	0.0000	0.0000
131.0000			
response_body_len	0.0000	0.0000	0.0000
5242880.0000			
ct_srv_src	2.0000	5.0000	11.0000
63.0000			
ct_state_ttl	1.0000	1.0000	2.0000
6.0000			
ct_dst_ltm	1.0000	2.0000	6.0000
59.0000			
ct_src_dport_ltm	1.0000	1.0000	4.0000
59.0000			
ct_dst_sport_ltm	1.0000	1.0000	3.0000
38.0000			
ct_dst_src_ltm	1.0000	3.0000	6.0000
63.0000			

is_ftp_login	0.0000	0.0000	0.0000
2.0000			
ct_ftp_cmd	0.0000	0.0000	0.0000
2.0000			
ct_flw_http_mthd	0.0000	0.0000	0.0000
16.0000			
ct_src_ltm	1.0000	3.0000	7.0000
60.0000			
ct_srv_dst	2.0000	5.0000	11.0000
62.0000			
is_sm_ips_ports	0.0000	0.0000	0.0000
1.0000			

```
numerical_summary[
  ["min", "max", "mean", "std", "50%"]
].sort_values("max", ascending=False)
```

	min	max	mean
std \			
sload	0.0000	5268000256.0000	64549016.9141
179861832.6300			
stcpb	0.0000	4294949667.0000	1084641551.1153
1390859761.6104			
dtcpb	0.0000	4294880717.0000	1073464670.8804
1381996192.0328			
dload	0.0000	20821108.0000	630546.9590
2393000.5556			
dbytes	0.0000	14657531.0000	13233.7856
151471.4561			
sbytes	24.0000	14355774.0000	7993.9082
171642.2619			
response_body_len	0.0000	5242880.0000	1595.3719
38066.9723			
sjit	0.0000	1483830.9170	6363.0751
56724.0167			
rate	0.0000	1000000.0030	82410.8867
148620.3670			
djit	0.0000	463199.2401	535.1804
3635.3054			
sinpkt	0.0000	60009.9920	755.3943
6182.6157			
dinpkt	0.0000	57739.2400	121.7013
1292.3785			
dpkts	0.0000	11018.0000	17.5459
115.5741			
spkts	1.0000	10646.0000	18.6665
133.9164			
dloss	0.0000	5507.0000	6.3086
55.7080			
sloss	0.0000	5319.0000	4.7537

64.6496			
smean	24.0000	1504.0000	139.5286
208.4721			
dmean	0.0000	1500.0000	116.2751
244.6003			
sttl	0.0000	255.0000	180.9677
101.5134			
swin	0.0000	255.0000	133.4591
127.3570			
dwin	0.0000	255.0000	128.2866
127.4914			
dttl	0.0000	253.0000	95.7130
116.6677			
trans_depth	0.0000	131.0000	0.0943
0.5429			
ct_dst_src_ltm	1.0000	63.0000	7.4564
11.4152			
ct_srv_src	1.0000	63.0000	9.5466
11.0903			
ct_srv_dst	1.0000	62.0000	9.1643
11.1214			
ct_src_ltm	1.0000	60.0000	6.4684
8.5439			
dur	0.0000	60.0000	1.0068
4.7104			
ct_dst_ltm	1.0000	59.0000	5.7449
8.4181			
ct_src_dport_ltm	1.0000	59.0000	4.9289
8.3895			
ct_dst_sport_ltm	1.0000	38.0000	3.6630
5.9154			
ct_flw_http_mthd	0.0000	16.0000	0.1297
0.6387			
ct_state_ttl	0.0000	6.0000	1.3693
1.0672			
tcprtt	0.0000	3.8215	0.0559
0.1160			
synack	0.0000	3.2268	0.0293
0.0709			
ackdat	0.0000	2.9288	0.0267
0.0551			
is_ftp_login	0.0000	2.0000	0.0083
0.0912			
ct_ftp_cmd	0.0000	2.0000	0.0084
0.0925			
is_sm_ips_ports	0.0000	1.0000	0.0111
0.1049			

50%

sload	577003.2188
stcpb	27888855.0000
dtcpb	28569748.5000
dload	2112.9514
dbytes	178.0000
sbytes	534.0000
response_body_len	0.0000
sjit	17.6239
rate	2650.1767
djit	0.0000
sinpkt	0.5579
dinpkt	0.0100
dpkts	2.0000
spkts	6.0000
dloss	0.0000
sloss	1.0000
smean	65.0000
dmean	44.0000
sttl	254.0000
swin	255.0000
dwin	255.0000
dttl	29.0000
trans_depth	0.0000
ct_dst_src_ltm	3.0000
ct_srv_src	5.0000
ct_srv_dst	5.0000
ct_src_ltm	3.0000
dur	0.0141
ct_dst_ltm	2.0000
ct_src_dport_ltm	1.0000
ct_dst_sport_ltm	1.0000
ct_flw_http_mthd	0.0000
ct_state_ttl	1.0000
tcprrt	0.0006
synack	0.0004
ackdat	0.0001
is_ftp_login	0.0000
ct_ftp_cmd	0.0000
is_sm_ips_ports	0.0000

check skewness

```
skewness = train_df[numerical_features].skew()
```

```
skewness.sort_values(
    key=lambda x: x.abs(),
    ascending=False
)
```

trans_depth	170.7944
response_body_len	74.6352

djit	60.5623
dloss	54.4656
sbytes	53.7785
dbytes	52.5504
sloss	52.4653
dpkts	49.3041
spkts	47.7478
dinpkt	23.0130
sjit	15.3494
synack	13.8097
ct_flw_http_mthd	12.5577
ct_ftp_cmd	11.2432
is_ftp_login	11.0423
ackdat	9.6287
dur	9.5238
sload	9.4884
is_sm_ips_ports	9.3218
sinpkt	9.2741
tcprtt	9.0603
dload	4.7539
smean	3.5441
rate	3.4995
dmean	3.1068
ct_src_dport_ltm	2.8568
ct_dst_ltm	2.6670
ct_dst_sport_ltm	2.5068
ct_src_ltm	2.4373
ct_dst_src_ltm	2.1890
ct_srv_dst	1.8989
ct_srv_src	1.8564
ct_state_ttl	1.5222
dtcpb	0.9301
stcpb	0.9164
sttl	-0.6904
dttl	0.5752
swin	-0.0936
dwin	-0.0123

dtype: float64

identifying highly skewed features

```
highly_skewed = skewness[
    skewness.abs() > 1
].sort_values(
    key=lambda x: x.abs(),
    ascending=False
)

print(
    f"Highly skewed features: {len(highly_skewed)}"
)
```

highly_skewed

Highly skewed features: 33

trans_depth	170.7944
response_body_len	74.6352
djit	60.5623
dloss	54.4656
sbytes	53.7785
dbytes	52.5504
sloss	52.4653
dpkts	49.3041
spkts	47.7478
dinpkt	23.0130
sjit	15.3494
synack	13.8097
ct_flw_http_mthd	12.5577
ct_ftp_cmd	11.2432
is_ftp_login	11.0423
ackdat	9.6287
dur	9.5238
sload	9.4884
is_sm_ips_ports	9.3218
sinpkt	9.2741
tcprtt	9.0603
dload	4.7539
smean	3.5441
rate	3.4995
dmean	3.1068
ct_src_dport_ltm	2.8568
ct_dst_ltm	2.6670
ct_dst_sport_ltm	2.5068
ct_src_ltm	2.4373
ct_dst_src_ltm	2.1890
ct_srv_dst	1.8989
ct_srv_src	1.8564
ct_state_ttl	1.5222

dtype: float64

zero heavy features

```
zero_percentage = (  
    (train_df[numerical_features] == 0)  
    .mean()  
    * 100  
) .sort_values(ascending=False)
```

zero_percentage

is_ftp_login	99.1765
ct_ftp_cmd	99.1741
is_sm_ips_ports	98.8874
response_body_len	93.1290
ct_flw_http_mthd	90.7934
trans_depth	90.7909
djit	50.4688
dloss	50.0462
ackdat	49.9927
synack	49.9526
tcprtt	49.9526
dwin	49.6818
dtcpb	49.6696
stcpb	49.6538
sloss	47.7579
swin	47.6570
sjit	46.9441
dttl	43.8068
dinpkt	43.7436
dload	43.7412
dmean	43.7327
dpkts	43.7327
dbytes	43.7327
ct_state_ttl	20.7028
sttl	1.2729
sload	1.2134
rate	1.2134
dur	1.1539
sinpkt	0.2198
sbytes	0.0000
spkts	0.0000
ct_src_dport_ltm	0.0000
ct_dst_ltm	0.0000
ct_srv_src	0.0000
smean	0.0000
ct_dst_sport_ltm	0.0000
ct_dst_src_ltm	0.0000
ct_src_ltm	0.0000
ct_srv_dst	0.0000

dtype: float64

```
zero_heavy_features = zero_percentage[
    zero_percentage > 90
]
```

zero_heavy_features

is_ftp_login	99.1765
ct_ftp_cmd	99.1741
is_sm_ips_ports	98.8874

```
response_body_len    93.1290
ct_flw_http_mthd     90.7934
trans_depth          90.7909
dtype: float64
```

Relationship with the Target

```
# normal vs attack statistically
normal_df = train_df[train_df["label"] == 0]
attack_df = train_df[train_df["label"] == 1]
```

```
print("Normal traffic:", normal_df.shape)
print("Attack traffic:", attack_df.shape)
```

```
Normal traffic: (37000, 45)
Attack traffic: (45332, 45)
```

```
# median
```

```
comparison = pd.DataFrame({
    "normal_median": normal_df[numerical_features].median(),
    "attack_median": attack_df[numerical_features].median(),
    "normal_mean": normal_df[numerical_features].mean(),
    "attack_mean": attack_df[numerical_features].mean()
})
```

```
comparison
```

	normal_median	attack_median	normal_mean \
dur	0.2228	0.0000	1.0127
spkts	10.0000	2.0000	22.7770
dpkts	8.0000	0.0000	25.4153
sbytes	974.0000	200.0000	4072.3774
dbytes	354.0000	0.0000	18704.9302
rate	118.6255	100000.0025	28349.9867
sttl	62.0000	254.0000	124.3190
dttl	29.0000	0.0000	108.4447
sload	91269.6836	50666664.0000	39753387.6737
dload	7679.0217	0.0000	1373613.4453
sloss	2.0000	0.0000	4.2986
dloss	1.0000	0.0000	9.0463
sinpkt	15.8369	0.0100	1581.8591
dinpkt	1.2095	0.0000	175.4665
sjit	638.1981	0.0000	8083.2278
djit	27.0856	0.0000	644.3505
swin	255.0000	0.0000	191.8910
stcpb	1251599584.0000	0.0000	1517902766.5889
dtcpb	1233811738.5000	0.0000	1506155577.3207
dwin	255.0000	0.0000	180.3951

tcprtt	0.0007	0.0000	0.0750
synack	0.0006	0.0000	0.0410
ackdat	0.0001	0.0000	0.0340
smean	73.0000	57.0000	153.6381
dmean	76.0000	0.0000	173.5489
trans_depth	0.0000	0.0000	0.1098
response_body_len	0.0000	0.0000	2286.9884
ct_srv_src	4.0000	7.0000	5.9843
ct_state_ttl	1.0000	2.0000	0.9930
ct_dst_ltm	2.0000	3.0000	3.3410
ct_src_dport_ltm	1.0000	2.0000	1.7575
ct_dst_sport_ltm	1.0000	2.0000	1.0854
ct_dst_src_ltm	2.0000	4.0000	3.9186
is_ftp_login	0.0000	0.0000	0.0099
ct_ftp_cmd	0.0000	0.0000	0.0101
ct_flw_http_mthd	0.0000	0.0000	0.1828
ct_src_ltm	2.0000	4.0000	3.8535
ct_srv_dst	4.0000	7.0000	5.5583
is_sm_ips_ports	0.0000	0.0000	0.0248

	attack_mean
dur	1.0019
spkts	15.3115
dpkts	11.1229
sbytes	11194.6635
dbytes	8768.2347
rate	126535.4191
sttl	227.2044
dttl	85.3214
sload	84787221.3148
dload	24055.7388
sloss	5.1251
dloss	4.0740
sinpkt	80.8333
dinpkt	77.8181
sjit	4959.0856
djit	446.0758
swin	85.7669
stcpb	731013540.6034
dtcpb	720302146.8733
dwin	85.7556
tcprtt	0.0403
synack	0.0197
ackdat	0.0207
smean	128.0124
dmean	69.5282
trans_depth	0.0816
response_body_len	1030.8742
ct_srv_src	12.4542

```

ct_state_ttl          1.6764
ct_dst_ltm            7.7070
ct_src_dport_ltm      7.5174
ct_dst_sport_ltm      5.7668
ct_dst_src_ltm        10.3438
is_ftp_login          0.0069
ct_ftp_cmd            0.0069
ct_flw_http_mthd      0.0865
ct_src_ltm            8.6026
ct_srv_dst            12.1075
is_sm_ips_ports       0.0000

```

```

# absolute standardized mean difference
feature_separation = []

```

```

for col in numerical_features:

```

```

    normal_mean = normal_df[col].mean()
    attack_mean = attack_df[col].mean()

```

```

    pooled_std = train_df[col].std()

```

```

    if pooled_std != 0:
        separation = abs(
            attack_mean - normal_mean
        ) / pooled_std

```

```

    else:
        separation = 0

```

```

    feature_separation.append(
        (col, separation)
    )

```

```

feature_separation_df = pd.DataFrame(
    feature_separation,
    columns=["feature", "separation_score"]
)

```

```

feature_separation_df = feature_separation_df.sort_values(
    "separation_score",
    ascending=False
)

```

```

feature_separation_df.head(20)

```

	feature	separation_score
6	sttl	1.0135
16	swin	0.8333
31	ct_dst_sport_ltm	0.7914
19	dwin	0.7423

30	ct_src_dport_ltm	0.6865
5	rate	0.6606
28	ct_state_ttl	0.6403
37	ct_srv_dst	0.5889
27	ct_srv_src	0.5834
18	dtcpb	0.5686
17	stcpb	0.5658
9	dload	0.5640
32	ct_dst_src_ltm	0.5629
36	ct_src_ltm	0.5558
29	ct_dst_ltm	0.5187
24	dmean	0.4253
21	synack	0.3014
20	tcprrt	0.2991
8	sload	0.2504
12	sinpkt	0.2428

Top 10 features

```
top_features = feature_separation_df.head(10)["feature"].tolist()
```

```
top_features
```

```
['sttl',
 'swin',
 'ct_dst_sport_ltm',
 'dwin',
 'ct_src_dport_ltm',
 'rate',
 'ct_state_ttl',
 'ct_srv_dst',
 'ct_srv_src',
 'dtcpb']
```

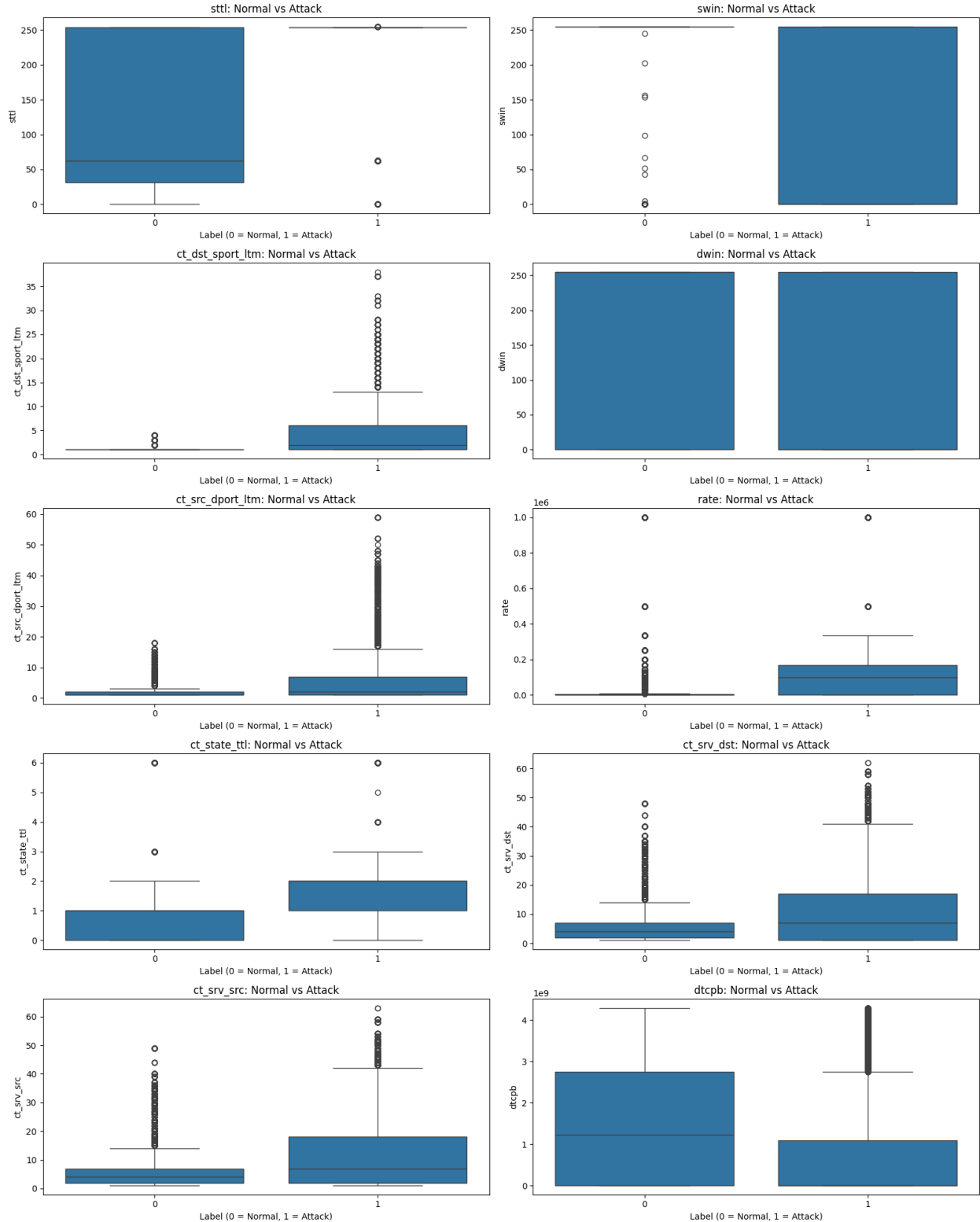
```
fig, axes = plt.subplots(
    5,
    2,
    figsize=(16, 20)
)
```

```
axes = axes.flatten()
```

```
for ax, feature in zip(axes, top_features):
```

```
    sns.boxplot(
        data=train_df,
        x="label",
        y=feature,
        ax=ax
    )
```

```
ax.set_title(  
    f"{feature}: Normal vs Attack"  
)  
  
ax.set_xlabel("Label (0 = Normal, 1 = Attack)")  
  
plt.tight_layout()  
plt.show()
```



```
# compare categorical features with attack label
for col in categorical_features:
```

```

attack_rate = (
    train_df
    .groupby(col)["label"]
    .agg(["count", "mean"])
    .rename(
        columns={"mean": "attack_rate"}
    )
    .sort_values(
        "attack_rate",
        ascending=False
    )
)

print(f"\n{'='*60}")
print(f"ATTACK RATE BY {col.upper()}")
print(f"{'='*60}")

display(attack_rate.head(20))

```

```

=====
ATTACK RATE BY PROTO
=====

```

	count	attack_rate
proto		
3pc	32	1.0000
a/n	32	1.0000
aes-sp3-d	32	1.0000
any	96	1.0000
argus	33	1.0000
aris	32	1.0000
ax.25	32	1.0000
bna	32	1.0000
bbn-rcc	34	1.0000
br-sat-mon	32	1.0000
cbt	33	1.0000
cphb	32	1.0000
cftp	32	1.0000
chaos	33	1.0000
compaq-peer	32	1.0000
crtp	32	1.0000
cpnx	32	1.0000
crudp	32	1.0000
dcn	33	1.0000
etherip	32	1.0000

ATTACK RATE BY SERVICE

=====

	count	attack_rate
service		
dhcp	26	1.0000
snmp	29	1.0000
ssl	30	1.0000
pop3	423	1.0000
irc	5	1.0000
dns	21367	0.8564
radius	9	0.7778
smtp	1851	0.6569
http	8287	0.5157
ftp	1552	0.5116
-	47153	0.4194
ftp-data	1396	0.3202
ssh	204	0.0196

=====

ATTACK RATE BY STATE

=====

	count	attack_rate
state		
CLO	1	1.0000
INT	34163	0.8687
ACC	4	0.5000
FIN	39339	0.3855
REQ	1842	0.0733
CON	6982	0.0500
RST	1	0.0000

correlation analysis

```
correlation_matrix = train_df[numerical_features].corr()
```

```
correlation_matrix
```

	dur	spkts	dpkts	sbytes	dbytes	rate
sttl \						
dur	1.0000	0.2802	0.2175	0.2254	0.1725	-0.1180
0.0010						
spkts	0.2802	1.0000	0.3696	0.9657	0.1983	-0.0682
0.0925						
dpkts	0.2175	0.3696	1.0000	0.1758	0.9764	-0.0832
0.1638						
sbytes	0.2254	0.9657	0.1758	1.0000	0.0100	-0.0251
0.0179						
dbytes	0.1725	0.1983	0.9764	0.0100	1.0000	-0.0480
0.1145						

rate	-0.1180	-0.0682	-0.0832	-0.0251	-0.0480	1.0000	
0.3882							
sttl	-0.0010	-0.0925	-0.1638	-0.0179	-0.1145	0.3882	
1.0000							
dttl	0.0900	0.0546	0.0365	0.0499	0.0125	-0.4539	-
0.0333							
sload	-0.0763	-0.0442	-0.0541	-0.0152	-0.0313	0.5501	
0.2529							
dload	-0.0470	0.0744	0.1338	-0.0064	0.1009	-0.1384	-
0.3862							
sloss	0.2401	0.9736	0.1891	0.9950	0.0146	-0.0401	-
0.0381							
dloss	0.1712	0.1987	0.9815	0.0071	0.9971	-0.0621	-
0.1377							
sinpkt	0.0798	-0.0145	-0.0171	-0.0054	-0.0102	-0.0657	-
0.1793							
dinpkt	0.1508	-0.0033	-0.0072	-0.0014	-0.0073	-0.0522	-
0.0062							
sjit	0.1466	-0.0024	-0.0039	-0.0027	-0.0052	-0.0620	
0.0301							
djit	0.1654	0.0105	0.0343	-0.0031	0.0292	-0.0816	-
0.0841							
swin	0.0840	0.1118	0.1415	0.0414	0.0830	-0.5341	-
0.3705							
stcpb	0.0445	0.0836	0.1103	0.0287	0.0645	-0.4285	-
0.3057							
dtcpb	0.0453	0.0862	0.1073	0.0331	0.0613	-0.4270	-
0.3048							
dwin	0.0612	0.1162	0.1475	0.0433	0.0865	-0.5531	-
0.3984							
tcprtt	0.0753	0.0099	-0.0070	0.0168	-0.0126	-0.2672	-
0.0180							
synack	0.0665	0.0042	-0.0110	0.0107	-0.0148	-0.2289	-
0.0189							
ackdat	0.0731	0.0155	-0.0006	0.0216	-0.0075	-0.2683	-
0.0136							
smean	0.0905	0.1930	0.0125	0.2067	-0.0305	-0.1098	
0.0011							
dmean	0.0436	0.1410	0.3960	-0.0032	0.3754	-0.2593	-
0.5320							
trans_depth	0.0060	0.0068	0.0267	0.0003	0.0285	-0.0961	-
0.1243							
response_body_len	0.0762	0.0856	0.3580	0.0017	0.3827	-0.0232	-
0.0549							
ct_srv_src	-0.0883	-0.0587	-0.0628	-0.0302	-0.0343	0.3677	
0.3115							
ct_state_ttl	0.0939	-0.0739	-0.1224	-0.0073	-0.0730	0.3995	
0.5561							
ct_dst_ltm	-0.0726	-0.0479	-0.0526	-0.0223	-0.0296	0.3110	

0.2207							
ct_src_dport_ltm	-0.0869	-0.0531	-0.0625	-0.0206	-0.0353	0.3456	
0.2876							
ct_dst_sport_ltm	-0.0816	-0.0551	-0.0675	-0.0205	-0.0390	0.3662	
0.3158							
ct_dst_src_ltm	-0.0711	-0.0619	-0.0750	-0.0241	-0.0440	0.3589	
0.3351							
is_ftp_login	0.0157	0.0104	0.0125	-0.0026	-0.0065	-0.0501	-
0.0896							
ct_ftp_cmd	0.0155	0.0104	0.0126	-0.0026	-0.0065	-0.0500	-
0.0899							
ct_flw_http_mthd	0.0189	0.0004	0.0210	-0.0046	0.0229	-0.1124	-
0.1748							
ct_src_ltm	-0.0808	-0.0494	-0.0574	-0.0215	-0.0333	0.3276	
0.2369							
ct_srv_dst	-0.0913	-0.0588	-0.0632	-0.0300	-0.0344	0.3708	
0.3041							
is_sm_ips_ports	0.0275	-0.0140	-0.0161	-0.0049	-0.0093	-0.0587	-
0.1879							

	dttl	sload	dload	sloss	dloss	sinpkt	
dinpkt \							
dur	0.0900	-0.0763	-0.0470	0.2401	0.1712	0.0798	
0.1508							
spkts	0.0546	-0.0442	0.0744	0.9736	0.1987	-0.0145	-
0.0033							
dpkts	0.0365	-0.0541	0.1338	0.1891	0.9815	-0.0171	-
0.0072							
sbytes	0.0499	-0.0152	-0.0064	0.9950	0.0071	-0.0054	-
0.0014							
dbytes	0.0125	-0.0313	0.1009	0.0146	0.9971	-0.0102	-
0.0073							
rate	-0.4539	0.5501	-0.1384	-0.0401	-0.0621	-0.0657	-
0.0522							
sttl	-0.0333	0.2529	-0.3862	-0.0381	-0.1377	-0.1793	-
0.0062							
dttl	1.0000	-0.2939	-0.1395	0.0612	0.0220	-0.0756	
0.0907							
sload	-0.2939	1.0000	-0.0928	-0.0259	-0.0405	-0.0417	-
0.0338							
dload	-0.1395	-0.0928	1.0000	0.0092	0.1173	-0.0321	-
0.0245							
sloss	0.0612	-0.0259	0.0092	1.0000	0.0147	-0.0081	-
0.0005							
dloss	0.0220	-0.0405	0.1173	0.0147	1.0000	-0.0131	-
0.0078							
sinpkt	-0.0756	-0.0417	-0.0321	-0.0081	-0.0131	1.0000	
0.1309							
dinpkt	0.0907	-0.0338	-0.0245	-0.0005	-0.0078	0.1309	

1.0000							
sjit	0.1406	-0.0400	-0.0288	-0.0004	-0.0053	0.0800	
0.7176							
djit	0.1138	-0.0528	-0.0313	0.0021	0.0309	0.0404	
0.4445							
swin	0.7499	-0.3615	0.2228	0.0702	0.1081	-0.0998	
0.0597							
stcpb	0.6098	-0.2783	0.1764	0.0498	0.0846	-0.0766	
0.0546							
dtcpb	0.6087	-0.2772	0.1820	0.0535	0.0814	-0.0770	
0.0488							
dwin	0.7820	-0.3591	0.2333	0.0713	0.1126	-0.0995	
0.0635							
tcprtt	0.6439	-0.1729	-0.1208	0.0210	-0.0099	-0.0401	
0.0757							
synack	0.5508	-0.1481	-0.1034	0.0143	-0.0128	-0.0340	
0.0658							
ackdat	0.6476	-0.1736	-0.1213	0.0259	-0.0043	-0.0408	
0.0747							
smean	0.1684	0.2547	-0.0838	0.2068	-0.0255	-0.0479	-
0.0075							
dmean	0.1083	-0.1696	0.5409	0.0155	0.3824	-0.0528	-
0.0186							
trans_depth	0.1629	-0.0622	-0.0321	0.0002	0.0261	-0.0184	-
0.0011							
response_body_len	0.0054	-0.0150	0.0255	-0.0018	0.3704	-0.0048	-
0.0034							
ct_srv_src	-0.4447	0.1586	-0.0542	-0.0401	-0.0459	-0.0802	-
0.0435							
ct_state_ttl	-0.2302	0.2333	-0.3324	-0.0247	-0.0937	0.0724	-
0.0323							
ct_dst_ltm	-0.3769	0.0903	-0.0697	-0.0311	-0.0388	-0.0595	-
0.0390							
ct_src_dport_ltm	-0.3499	0.1124	-0.1121	-0.0320	-0.0465	-0.0519	-
0.0330							
ct_dst_sport_ltm	-0.3578	0.0834	-0.1183	-0.0325	-0.0505	-0.0483	-
0.0299							
ct_dst_src_ltm	-0.3642	0.1554	-0.1317	-0.0358	-0.0565	-0.0588	-
0.0367							
is_ftp_login	0.0522	-0.0326	-0.0189	0.0054	0.0070	-0.0100	-
0.0031							
ct_ftp_cmd	0.0509	-0.0325	-0.0187	0.0054	0.0070	-0.0100	-
0.0031							
ct_flw_http_mthd	0.2086	-0.0728	-0.0411	-0.0046	0.0209	-0.0168	
0.0270							
ct_src_ltm	-0.4033	0.1100	-0.0660	-0.0318	-0.0431	-0.0681	-
0.0434							
ct_srv_dst	-0.4546	0.1603	-0.0590	-0.0403	-0.0461	-0.0771	-
0.0450							

is_sm_ips_ports	-0.0870	-0.0380	-0.0279	-0.0078	-0.0120	0.9445	-
0.0100							
	sjit	djit	swin	stcpb	dtcpb	dwin	
tcprtt \							
dur	0.1466	0.1654	0.0840	0.0445	0.0453	0.0612	
0.0753							
spkts	-0.0024	0.0105	0.1118	0.0836	0.0862	0.1162	
0.0099							
dpkts	-0.0039	0.0343	0.1415	0.1103	0.1073	0.1475	-
0.0070							
sbytes	-0.0027	-0.0031	0.0414	0.0287	0.0331	0.0433	
0.0168							
dbytes	-0.0052	0.0292	0.0830	0.0645	0.0613	0.0865	-
0.0126							
rate	-0.0620	-0.0816	-0.5341	-0.4285	-0.4270	-0.5531	-
0.2672							
sttl	0.0301	-0.0841	-0.3705	-0.3057	-0.3048	-0.3984	-
0.0180							
dttl	0.1406	0.1138	0.7499	0.6098	0.6087	0.7820	
0.6439							
sload	-0.0400	-0.0528	-0.3615	-0.2783	-0.2772	-0.3591	-
0.1729							
dload	-0.0288	-0.0313	0.2228	0.1764	0.1820	0.2333	-
0.1208							
sloss	-0.0004	0.0021	0.0702	0.0498	0.0535	0.0713	
0.0210							
dloss	-0.0053	0.0309	0.1081	0.0846	0.0814	0.1126	-
0.0099							
sinpkt	0.0800	0.0404	-0.0998	-0.0766	-0.0770	-0.0995	-
0.0401							
dinpkt	0.7176	0.4445	0.0597	0.0546	0.0488	0.0635	
0.0757							
sjit	1.0000	0.5923	0.1059	0.0891	0.0822	0.1095	
0.1127							
djit	0.5923	1.0000	0.1373	0.1172	0.1089	0.1432	
0.0633							
swin	0.1059	0.1373	1.0000	0.7441	0.7412	0.9601	
0.4597							
stcpb	0.0891	0.1172	0.7441	1.0000	0.6245	0.7737	
0.3760							
dtcpb	0.0822	0.1089	0.7412	0.6245	1.0000	0.7713	
0.3785							
dwin	0.1095	0.1432	0.9601	0.7737	0.7713	1.0000	
0.4783							
tcprtt	0.1127	0.0633	0.4597	0.3760	0.3785	0.4783	
1.0000							
synack	0.1000	0.0529	0.3939	0.3217	0.3264	0.4096	
0.9395							

ackdat	0.1087	0.0652	0.4615	0.3781	0.3773	0.4805
0.8977						
smean	-0.0082	-0.0180	0.1395	0.1175	0.1201	0.1574
0.1016						
dmean	-0.0111	0.1436	0.4095	0.3303	0.3259	0.4283 -
0.0082						
trans_depth	0.0048	0.0629	0.1657	0.1317	0.1354	0.1726
0.0646						
response_body_len	-0.0028	0.0106	0.0400	0.0301	0.0299	0.0417 -
0.0072						
ct_srv_src	-0.0463	-0.0775	-0.4617	-0.3692	-0.3676	-0.4763 -
0.2640						
ct_state_ttl	-0.0248	-0.0853	-0.3616	-0.4121	-0.4084	-0.5352 -
0.0067						
ct_dst_ltm	-0.0445	-0.0591	-0.3856	-0.3090	-0.3072	-0.3965 -
0.2297						
ct_src_dport_ltm	-0.0388	-0.0595	-0.4336	-0.3220	-0.3213	-0.4154 -
0.2064						
ct_dst_sport_ltm	-0.0356	-0.0575	-0.4592	-0.3422	-0.3414	-0.4422 -
0.2099						
ct_dst_src_ltm	-0.0409	-0.0674	-0.4236	-0.3491	-0.3477	-0.4497 -
0.2090						
is_ftp_login	-0.0016	0.0483	0.0867	0.0718	0.0639	0.0903
0.0076						
ct_ftp_cmd	-0.0017	0.0475	0.0865	0.0721	0.0631	0.0901
0.0070						
ct_flw_http_mthd	0.0579	0.0852	0.1939	0.1559	0.1634	0.2019
0.0959						
ct_src_ltm	-0.0493	-0.0659	-0.4136	-0.3269	-0.3254	-0.4197 -
0.2466						
ct_srv_dst	-0.0488	-0.0781	-0.4728	-0.3762	-0.3750	-0.4839 -
0.2716						
is_sm_ips_ports	-0.0119	-0.0156	-0.1102	-0.0822	-0.0824	-0.1067 -
0.0511						

	synack	ackdat	smean	dmean	trans_depth	\
dur	0.0665	0.0731	0.0905	0.0436	0.0060	
spkts	0.0042	0.0155	0.1930	0.1410	0.0068	
dpkts	-0.0110	-0.0006	0.0125	0.3960	0.0267	
sbytes	0.0107	0.0216	0.2067	-0.0032	0.0003	
dbytes	-0.0148	-0.0075	-0.0305	0.3754	0.0285	
rate	-0.2289	-0.2683	-0.1098	-0.2593	-0.0961	
sttl	-0.0189	-0.0136	0.0011	-0.5320	-0.1243	
dttl	0.5508	0.6476	0.1684	0.1083	0.1629	
sload	-0.1481	-0.1736	0.2547	-0.1696	-0.0622	
dload	-0.1034	-0.1213	-0.0838	0.5409	-0.0321	
sloss	0.0143	0.0259	0.2068	0.0155	0.0002	
dloss	-0.0128	-0.0043	-0.0255	0.3824	0.0261	
sinpkt	-0.0340	-0.0408	-0.0479	-0.0528	-0.0184	

dinpkt	0.0658	0.0747	-0.0075	-0.0186	-0.0011
sjit	0.1000	0.1087	-0.0082	-0.0111	0.0048
djit	0.0529	0.0652	-0.0180	0.1436	0.0629
swin	0.3939	0.4615	0.1395	0.4095	0.1657
stcpb	0.3217	0.3781	0.1175	0.3303	0.1317
dtcpb	0.3264	0.3773	0.1201	0.3259	0.1354
dwin	0.4096	0.4805	0.1574	0.4283	0.1726
tcprtt	0.9395	0.8977	0.1016	-0.0082	0.0646
synack	1.0000	0.6924	0.0851	-0.0169	0.0471
ackdat	0.6924	1.0000	0.1044	0.0044	0.0754
smean	0.0851	0.1044	1.0000	-0.0846	0.0220
dmean	-0.0169	0.0044	-0.0846	1.0000	0.1270
trans_depth	0.0471	0.0754	0.0220	0.1270	1.0000
response_body_len	-0.0082	-0.0047	-0.0167	0.2084	0.0698
ct_srv_src	-0.2232	-0.2689	-0.1448	-0.2019	-0.1136
ct_state_ttl	0.0070	-0.0231	-0.0750	-0.4099	-0.0939
ct_dst_ltm	-0.1957	-0.2320	-0.1391	-0.1674	-0.0717
ct_src_dport_ltm	-0.1760	-0.2082	-0.1373	-0.2014	-0.0756
ct_dst_sport_ltm	-0.1796	-0.2109	-0.1687	-0.2114	-0.0752
ct_dst_src_ltm	-0.1767	-0.2128	-0.1181	-0.2357	-0.0837
is_ftp_login	0.0025	0.0127	-0.0281	-0.0170	-0.0158
ct_ftp_cmd	0.0020	0.0120	-0.0282	-0.0170	-0.0157
ct_flw_http_mthd	0.0784	0.1011	-0.0088	0.1218	0.3452
ct_src_ltm	-0.2103	-0.2489	-0.1392	-0.1769	-0.0842
ct_srv_dst	-0.2298	-0.2764	-0.1464	-0.2042	-0.1105
is_sm_ips_ports	-0.0438	-0.0513	-0.0476	-0.0504	-0.0184
response_body_len ct_srv_src ct_state_ttl					
ct_dst_ltm \					
dur	0.0762	-0.0883	0.0939	-	
0.0726					
spkts	0.0856	-0.0587	-0.0739	-	
0.0479					
dpkts	0.3580	-0.0628	-0.1224	-	
0.0526					
sbytes	0.0017	-0.0302	-0.0073	-	
0.0223					
dbytes	0.3827	-0.0343	-0.0730	-	
0.0296					
rate	-0.0232	0.3677	0.3995		
0.3110					
sttl	-0.0549	0.3115	0.5561		
0.2207					
dttl	0.0054	-0.4447	-0.2302	-	
0.3769					
sload	-0.0150	0.1586	0.2333		
0.0903					
dload	0.0255	-0.0542	-0.3324	-	
0.0697					

sloss	-0.0018	-0.0401	-0.0247	-
0.0311				
dloss	0.3704	-0.0459	-0.0937	-
0.0388				
sinpkt	-0.0048	-0.0802	0.0724	-
0.0595				
dinpkt	-0.0034	-0.0435	-0.0323	-
0.0390				
sjit	-0.0028	-0.0463	-0.0248	-
0.0445				
djit	0.0106	-0.0775	-0.0853	-
0.0591				
swin	0.0400	-0.4617	-0.3616	-
0.3856				
stcpb	0.0301	-0.3692	-0.4121	-
0.3090				
dtcpb	0.0299	-0.3676	-0.4084	-
0.3072				
dwin	0.0417	-0.4763	-0.5352	-
0.3965				
tcprtt	-0.0072	-0.2640	-0.0067	-
0.2297				
synack	-0.0082	-0.2232	0.0070	-
0.1957				
ackdat	-0.0047	-0.2689	-0.0231	-
0.2320				
smean	-0.0167	-0.1448	-0.0750	-
0.1391				
dmean	0.2084	-0.2019	-0.4099	-
0.1674				
trans_depth	0.0698	-0.1136	-0.0939	-
0.0717				
response_body_len	1.0000	-0.0292	-0.0359	-
0.0154				
ct_srv_src	-0.0292	1.0000	0.3129	
0.8422				
ct_state_ttl	-0.0359	0.3129	1.0000	
0.2421				
ct_dst_ltm	-0.0154	0.8422	0.2421	
1.0000				
ct_src_dport_ltm	-0.0188	0.8555	0.2426	
0.9604				
ct_dst_sport_ltm	-0.0187	0.8012	0.2637	
0.8722				
ct_dst_src_ltm	-0.0205	0.9338	0.3629	
0.8682				
is_ftp_login	-0.0038	-0.0681	-0.0655	-
0.0290				
ct_ftp_cmd	-0.0038	-0.0679	-0.0659	-

0.0285				
ct_flw_http_mthd	0.0672	-0.1232	-0.1008	-
0.0714				
ct_src_ltm	-0.0179	0.8225	0.2404	
0.9323				
ct_srv_dst	-0.0279	0.9778	0.3058	
0.8552				
is_sm_ips_ports	-0.0044	-0.0796	0.0644	-
0.0570				
	ct_src_dport_ltm	ct_dst_sport_ltm	ct_dst_src_ltm	
\				
dur	-0.0869	-0.0816	-0.0711	
spkts	-0.0531	-0.0551	-0.0619	
dpkts	-0.0625	-0.0675	-0.0750	
sbytes	-0.0206	-0.0205	-0.0241	
dbytes	-0.0353	-0.0390	-0.0440	
rate	0.3456	0.3662	0.3589	
sttl	0.2876	0.3158	0.3351	
dttl	-0.3499	-0.3578	-0.3642	
sload	0.1124	0.0834	0.1554	
dload	-0.1121	-0.1183	-0.1317	
sloss	-0.0320	-0.0325	-0.0358	
dloss	-0.0465	-0.0505	-0.0565	
sinpkt	-0.0519	-0.0483	-0.0588	
dinpkt	-0.0330	-0.0299	-0.0367	
sjit	-0.0388	-0.0356	-0.0409	
djit	-0.0595	-0.0575	-0.0674	
swin	-0.4336	-0.4592	-0.4236	
stcpb	-0.3220	-0.3422	-0.3491	
dtcpb	-0.3213	-0.3414	-0.3477	
dwin	-0.4154	-0.4422	-0.4497	

tcprtt	-0.2064	-0.2099	-0.2090	
synack	-0.1760	-0.1796	-0.1767	
ackdat	-0.2082	-0.2109	-0.2128	
smean	-0.1373	-0.1687	-0.1181	
dmean	-0.2014	-0.2114	-0.2357	
trans_depth	-0.0756	-0.0752	-0.0837	
response_body_len	-0.0188	-0.0187	-0.0205	
ct_srv_src	0.8555	0.8012	0.9338	
ct_state_ttl	0.2426	0.2637	0.3629	
ct_dst_ltm	0.9604	0.8722	0.8682	
ct_src_dport_ltm	1.0000	0.9116	0.8760	
ct_dst_sport_ltm	0.9116	1.0000	0.8310	
ct_dst_src_ltm	0.8760	0.8310	1.0000	
is_ftp_login	-0.0422	-0.0405	-0.0404	
ct_ftp_cmd	-0.0420	-0.0403	-0.0401	
ct_flw_http_mthd	-0.0788	-0.0738	-0.0867	
ct_src_ltm	0.9332	0.8473	0.8400	
ct_srv_dst	0.8636	0.8110	0.9410	
is_sm_ips_ports	-0.0469	-0.0438	-0.0580	
	is_ftp_login	ct_ftp_cmd	ct_flw_http_mthd	
ct_src_ltm \				
dur	0.0157	0.0155	0.0189	-
0.0808				
spkts	0.0104	0.0104	0.0004	-
0.0494				
dpkts	0.0125	0.0126	0.0210	-
0.0574				
sbytes	-0.0026	-0.0026	-0.0046	-
0.0215				
dbytes	-0.0065	-0.0065	0.0229	-

0.0333				
rate	-0.0501	-0.0500	-0.1124	
0.3276				
sttl	-0.0896	-0.0899	-0.1748	
0.2369				
dttl	0.0522	0.0509	0.2086	-
0.4033				
sload	-0.0326	-0.0325	-0.0728	
0.1100				
dload	-0.0189	-0.0187	-0.0411	-
0.0660				
sloss	0.0054	0.0054	-0.0046	-
0.0318				
dloss	0.0070	0.0070	0.0209	-
0.0431				
sinpkt	-0.0100	-0.0100	-0.0168	-
0.0681				
dinpkt	-0.0031	-0.0031	0.0270	-
0.0434				
sjit	-0.0016	-0.0017	0.0579	-
0.0493				
djit	0.0483	0.0475	0.0852	-
0.0659				
swin	0.0867	0.0865	0.1939	-
0.4136				
stcpb	0.0718	0.0721	0.1559	-
0.3269				
dtcpb	0.0639	0.0631	0.1634	-
0.3254				
dwin	0.0903	0.0901	0.2019	-
0.4197				
tcprtt	0.0076	0.0070	0.0959	-
0.2466				
synack	0.0025	0.0020	0.0784	-
0.2103				
ackdat	0.0127	0.0120	0.1011	-
0.2489				
smean	-0.0281	-0.0282	-0.0088	-
0.1392				
dmean	-0.0170	-0.0170	0.1218	-
0.1769				
trans_depth	-0.0158	-0.0157	0.3452	-
0.0842				
response_body_len	-0.0038	-0.0038	0.0672	-
0.0179				
ct_srv_src	-0.0681	-0.0679	-0.1232	
0.8225				
ct_state_ttl	-0.0655	-0.0659	-0.1008	
0.2404				
ct_dst_ltm	-0.0290	-0.0285	-0.0714	

0.9323				
ct_src_dport_ltm	-0.0422	-0.0420	-0.0788	
0.9332				
ct_dst_sport_ltm	-0.0405	-0.0403	-0.0738	
0.8473				
ct_dst_src_ltm	-0.0404	-0.0401	-0.0867	
0.8400				
is_ftp_login	1.0000	0.9943	-0.0185	-
0.0298				
ct_ftp_cmd	0.9943	1.0000	-0.0184	-
0.0294				
ct_flw_http_mthd	-0.0185	-0.0184	1.0000	-
0.0934				
ct_src_ltm	-0.0298	-0.0294	-0.0934	
1.0000				
ct_srv_dst	-0.0647	-0.0644	-0.1187	
0.8224				
is_sm_ips_ports	-0.0096	-0.0096	-0.0215	-
0.0652				

	ct_srv_dst	is_sm_ips_ports
dur	-0.0913	0.0275
spkts	-0.0588	-0.0140
dpkts	-0.0632	-0.0161
sbytes	-0.0300	-0.0049
dbytes	-0.0344	-0.0093
rate	0.3708	-0.0587
sttl	0.3041	-0.1879
dttl	-0.4546	-0.0870
sload	0.1603	-0.0380
dload	-0.0590	-0.0279
sloss	-0.0403	-0.0078
dloss	-0.0461	-0.0120
sinpkt	-0.0771	0.9445
dinpkt	-0.0450	-0.0100
sjit	-0.0488	-0.0119
djit	-0.0781	-0.0156
swin	-0.4728	-0.1102
stcpb	-0.3762	-0.0822
dtcpb	-0.3750	-0.0824
dwin	-0.4839	-0.1067
tcprrt	-0.2716	-0.0511
synack	-0.2298	-0.0438
ackdat	-0.2764	-0.0513
smean	-0.1464	-0.0476
dmean	-0.2042	-0.0504
trans_depth	-0.1105	-0.0184
response_body_len	-0.0279	-0.0044
ct_srv_src	0.9778	-0.0796

ct_state_ttl	0.3058	0.0644
ct_dst_ltm	0.8552	-0.0570
ct_src_dport_ltm	0.8636	-0.0469
ct_dst_sport_ltm	0.8110	-0.0438
ct_dst_src_ltm	0.9410	-0.0580
is_ftp_login	-0.0647	-0.0096
ct_ftp_cmd	-0.0644	-0.0096
ct_flw_http_mthd	-0.1187	-0.0215
ct_src_ltm	0.8224	-0.0652
ct_srv_dst	1.0000	-0.0757
is_sm_ips_ports	-0.0757	1.0000

highly correlated pairs

```
correlation_pairs = (
    correlation_matrix
    .abs()
    .where(
        np.triu(
            np.ones(correlation_matrix.shape),
            k=1
        ).astype(bool)
    )
    .stack()
    .sort_values(
        ascending=False
    )
)
```

```
high_correlation_pairs = correlation_pairs[
    correlation_pairs > 0.90
]
```

high_correlation_pairs

dbytes	dloss	0.9971
sbytes	sloss	0.9950
is_ftp_login	ct_ftp_cmd	0.9943
dpkts	dloss	0.9815
ct_srv_src	ct_srv_dst	0.9778
dpkts	dbytes	0.9764
spkts	sloss	0.9736
	sbytes	0.9657
ct_dst_ltm	ct_src_dport_ltm	0.9604
swin	dwin	0.9601
sinpkt	is_sm_ips_ports	0.9445
ct_dst_src_ltm	ct_srv_dst	0.9410
tcprtt	synack	0.9395
ct_srv_src	ct_dst_src_ltm	0.9338
ct_src_dport_ltm	ct_src_ltm	0.9332
ct_dst_ltm	ct_src_ltm	0.9323

```
ct_src_dport_ltm  ct_dst_sport_ltm    0.9116
dtype: float64
```

Preprocessing Pipeline Testing

```
# import and split
from src.data.preprocessing import UNSWPreprocessor

preprocessor = UNSWPreprocessor()

X_train, y_train, attack_train = (
    preprocessor.split_features_and_target(train_df)
)

X_test, y_test, attack_test = (
    preprocessor.split_features_and_target(test_df)
)

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("attack_train:", attack_train.shape)

print("\nX_test:", X_test.shape)
print("y_test:", y_test.shape)
print("attack_test:", attack_test.shape)

X_train: (82332, 42)
y_train: (82332,)
attack_train: (82332,)

X_test: (175341, 42)
y_test: (175341,)
attack_test: (175341,)

# preprocessing pipeline
X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print("Original train:", X_train.shape)
print("Processed train:", X_train_processed.shape)

print("\nOriginal test:", X_test.shape)
print("Processed test:", X_test_processed.shape)

Original train: (82332, 42)
Processed train: (82332, 190)
```

```
Original test: (175341, 42)
Processed test: (175341, 190)
```

```
# verify feature identification
print("Numerical features:")
print(preprocessor.numerical_features)

print("\nNumber of numerical features:")
print(len(preprocessor.numerical_features))

print("\nCategorical features:")
print(preprocessor.categorical_features)

print("\nNumber of categorical features:")
print(len(preprocessor.categorical_features))
```

```
Numerical features:
['dur', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl',
'sload', 'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit',
'djit', 'swin', 'stcpb', 'dtcpb', 'dwin', 'tcprrt', 'synack',
'ackdat', 'smean', 'dmean', 'trans_depth', 'response_body_len',
'ct_srv_src', 'ct_state_ttl', 'ct_dst_ltm', 'ct_src_dport_ltm',
'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'is_ftp_login', 'ct_ftp_cmd',
'ct_flw_http_mthd', 'ct_src_ltm', 'ct_srv_dst', 'is_sm_ips_ports']
```

```
Number of numerical features:
39
```

```
Categorical features:
['proto', 'service', 'state']
```

```
Number of categorical features:
3
```

```
# assertion
assert (
    X_train_processed.shape[1]
    ==
    X_test_processed.shape[1]
), "Train and test have different feature spaces!"
```

```
print("Train and test feature spaces match.")
```

```
Train and test feature spaces match.
```

```
#check generated feature names
feature_names = preprocessor.transformer.get_feature_names_out()

print("Number of processed features:", len(feature_names))

print("\nFirst 20:")
```

```

print(feature_names[:20])

print("\nLast 20:")
print(feature_names[-20:])

Number of processed features: 190

First 20:
['numeric__dur' 'numeric__spkts' 'numeric__dpkts' 'numeric__sbytes'
 'numeric__dbytes' 'numeric__rate' 'numeric__sttl' 'numeric__dttl'
 'numeric__sload' 'numeric__dload' 'numeric__sloss' 'numeric__dloss'
 'numeric__sinpkt' 'numeric__dinpkt' 'numeric__sjit' 'numeric__djit'
 'numeric__swin' 'numeric__stcpb' 'numeric__dtcpb' 'numeric__dwin']

Last 20:
['categorical__service_' 'categorical__service_dhcp'
 'categorical__service_dns' 'categorical__service_ftp'
 'categorical__service_ftp-data' 'categorical__service_http'
 'categorical__service_irc' 'categorical__service_pop3'
 'categorical__service_radius' 'categorical__service_smtp'
 'categorical__service_snmp' 'categorical__service_ssh'
 'categorical__service_ssl' 'categorical__state_ACC'
 'categorical__state_CLO' 'categorical__state_CON'
 'categorical__state_FIN' 'categorical__state_INT'
 'categorical__state_REQ' 'categorical__state_RST']

#sanity checks
assert X_train.shape[1] == 42
assert X_test.shape[1] == 42

assert "label" not in X_train.columns
assert "attack_cat" not in X_train.columns
assert "id" not in X_train.columns

assert len(preprocessor.numerical_features) == 39
assert len(preprocessor.categorical_features) == 3

print("All preprocessing sanity checks passed.")

All preprocessing sanity checks passed.

```

Summary of Exploratory Data Analysis

This exploratory analysis established the structure and key statistical characteristics of the UNSW-NB15 dataset and identified the major considerations required for downstream preprocessing and modelling.

Dataset Structure

The official training and testing splits contain:

- **Training set:** 82,332 records and 45 columns
- **Testing set:** 175,341 records and 45 columns

After separating the identifier (`id`), binary target (`label`), and attack category metadata (`attack_cat`), the model input consists of **42 features**.

These features are composed of:

- **39 numerical features**
- **3 categorical features:** `proto`, `service`, and `state`

The `attack_cat` column provides additional attack-type information and is retained as metadata rather than being used as an input feature for the binary intrusion-detection task.

Data Quality

No missing values were observed in either the training or testing split.

Additionally, no completely duplicated rows were found when considering all columns, including the record identifier. However, when the `id` column was excluded, a substantial number of duplicate feature records were observed:

- **Training set:** 26,387 duplicate feature rows
- **Testing set:** 67,601 duplicate feature rows

These duplicates were not removed at this stage because identical network-flow feature representations may legitimately occur multiple times and removing them could alter the original dataset distribution. Their potential impact on model evaluation will therefore be considered during later experiments rather than being handled through arbitrary deletion.

Class Distribution

The binary target distribution is relatively balanced:

- **Normal traffic (`label = 0`):** 37,000 records, approximately 44.94%
- **Attack traffic (`label = 1`):** 45,332 records, approximately 55.06%

Therefore, the binary classification task does not exhibit severe class imbalance. Nevertheless, evaluation should not rely solely on accuracy, and metrics such as precision, recall, F1-score, and class-wise performance should be considered during model benchmarking.

Attack Category Distribution

Although the binary target is reasonably balanced, the individual attack categories are highly unevenly distributed.

The largest attack categories are:

- **Generic:** 18,871 records
- **Exploits:** 11,132 records
- **Fuzzers:** 6,062 records
- **DoS:** 4,089 records
- **Reconnaissance:** 3,496 records

Several categories are extremely rare:

- **Analysis:** 677 records
- **Backdoor:** 583 records
- **Shellcode:** 378 records
- **Worms:** 44 records

This distinction is important: while the current binary classification task is not severely imbalanced, any future multi-class attack classification would face substantial class imbalance and would require additional consideration.

The analysis also confirmed that `attack_cat` is consistent with the binary label: `Normal` corresponds to `label = 0`, while the remaining attack categories correspond to `label = 1`.

Feature Characteristics

The numerical features exhibit large differences in scale, distribution, and cardinality. Several features span substantially different numerical ranges, indicating that scaling may be important for models that are sensitive to feature magnitude.

A large proportion of the numerical variables are also strongly right-skewed. Using an absolute skewness threshold greater than 1, **33 of the 39 numerical features** were identified as highly skewed.

Some of the most heavily skewed features include:

- `trans_depth`
- `response_body_len`
- `djit`
- `dloss`
- `sbytes`
- `dbytes`
- `sloss`
- `dpkts`

- `spkts`

These distributions indicate that scaling alone may not always be sufficient, and transformation strategies such as `log1p` should be experimentally evaluated.

Several numerical variables also contain relatively low numbers of unique values, meaning that not every feature should necessarily be treated as a continuous measurement in the same conceptual sense. This was taken into account during preprocessing design.

Feature Redundancy

Correlation analysis revealed several groups of strongly related numerical features. Examples include relationships between packet counts, byte counts, loss measurements, TCP window variables, timing variables, and connection history features.

The presence of highly correlated features suggests that the original feature space contains potential redundancy. This is particularly relevant for the future constrained QML stage, where reducing the dimensionality of the input space will be necessary.

However, high correlation alone does not prove that a feature is useless. Therefore, correlated features were not immediately removed during EDA. Instead, redundancy reduction was investigated as a separate preprocessing variant using a reproducible feature-selection criterion.

Key Conclusions

The EDA leads to the following conclusions:

1. The dataset is structurally clean, with no missing values and no complete duplicate rows.
2. The binary classification task is reasonably balanced, although the individual attack categories are strongly imbalanced.
3. The dataset contains both numerical and categorical features, requiring appropriate encoding of `proto`, `service`, and `state`.
4. Numerical features differ substantially in scale and exhibit widespread right-skewness.
5. Several strongly correlated feature groups indicate potential redundancy.
6. The dataset does not support a single obvious preprocessing strategy. Instead, multiple preprocessing representations should be experimentally compared before selecting the final pipeline.

These findings directly motivated the preprocessing experiments developed in the subsequent notebook.